Notes on the Stanford GraphBase

Adam Martinez*
University of Life

1 Project code organization

The main project has apparently been improved by a contributor other than D. E. Knuth, including patches for what seems like compatibility with post-ANSI C code, as well as better pointer handling practices. The changes beyond those in specific folders like `AMIGA` or `MSVC` (i.e. those included as patches in the project's root directory,) have not yet been inspected. Implementing the changes included in those files will not be done, as apparently the core logic was sound enough to even be implemented in both the Boost Graph Library, and in a subproject of it.

As per the project's `README`, all logic is implemented in terms of the *kernel routines*, so called because they implement the graph DSs in use as well as some routines for efficiently handling both random number generation and linked list traversal (maybe because adjacency lists are implemented in terms of linked lists?)

The actual logic for graph generation is stored in all `gb_*` files other than (1) `flip`, (2) `graph`, (3) `io`, and, (4) `sort`, except for (a) `dijk`, (b) `save`, and, (c) `types`. I believe the main approach to this should be an inspection of the kernel routines, followed by an in-depth study of the generative routines. Hopefully, this can yield some conclusions as to whether a generic interface over the graph kernel routines can be implemented, such that a different "backend" can be interchangebly used with the same generative routines.

1.1 Kernel code files

1.1.1 Random number generation (`gb_flip.w`)

In and of itself, this part of the program offers a function with which to initialize the random number generator, and a macro with which to produce a random number. Both of these are very much transparent in the way the perform their internal operations, as the initial routine expects an explicit seed with which (for now, I believe) the program "picks" a point in its deterministic sequence to start off producing values. Beyond this, the macro to be called makes explicit the fact that the generated numbers follow as part of, upon initialization, a predetermined series.

*`staying@never.land`

According to the file, the period of the numbers is of $2^{85} - 2^{30} = 2^{55}$. According to Skiena's book, the cycling of numbers that rely on 2^{32} calls of a linear congruential engine is worrying. Whether the algorithm used in this file is a linear congruential engine, and whether 2^{55} calls may be performed by today's computers in little more than 2^{32} calls is something I am not aware of.

Further inspection of the file reveals that this seems very much like an instance of a linear congruential engine, where the value of a random number n is determined as the function R_n, such that

$$\begin{aligned} &R_n = (R_{n-55} - R_{n-24}) \bmod m, \text{where } m \text{ is even and} \\ &R_0, R_1, \cdots, R_{54} \text{ is a series containing both even and odd numbers.} \end{aligned} \tag{1.1}$$

This looks a lot like the computation resolved in the example in [2, Sec. 16.7, p. 487]. It computes the value of the nth random number from some other $n - 55$th and $n - 24$th random numbers. This algorithm is also noted to consider m as taking on the largest value with which to bound the number the recurrence relation in the modulo's lhs resolves to, by taking on the 2^{31} full range of unsigned integer values.

In terms of the effectiveness of such a random number generator, D. E. Knuth notes that the chosen offsets, namely 24 and 55, should prove to be good enough for "most" applications. This likely doesn't cover cryptographic applications, but neither do I belive the Stanford GraphBase to require of fine-grained pseudorandom number generation. The point being, **the period of this generator should be at least $2^{55} - 1$.**

The C code generated after running `ctangle` is full of `#line` directives I'm going to ignore for the sake of sanity. Maybe there's more information on this in [3], [4]. The code then seems fairly straightforward.

The "public" interface in the header file exposes both the macro for advancing the random number sequence, as well as three more routines.

`gb_flip_cycle()`
: Yet unbeknownst to me.

`gb_init_rand()`
: To initialize the random number engine with a given seed.

`gb_unif_rand()`
: Yet unbeknownst to me.

Beyond this, the state of the whole shebang is kept through two globals; (1) an array `A` holding $n + 1$, where $n = 55$ elements, and, (2) a pointer to that array `gb_fptr`. The array is set to hold one more than the largest offset used in Equation (1.1), such that the first value is used as a sentinel (more on

this once I get to know the code better.) The pointer doesn't exist for any purpose other than allowing the user to run the macro exposed in the header file (which otherwise would not have any means of reaching for the array holding the core state.)

Of note is that the documentation speaks of a limitation in the `gb_next_rand` macro: There seems to be a test, referred to as the *birthday spacings test*, that fails to prove this to be a decent enough random number generator. The solution proposed by D. E. Knuth is to modify the definition of the macro such that instead of performing the following computation,

```
#define gb_next_rand() (*gb_fptr >= 0 ? *gb_fptr- : gb_flip_cycle())
```

it performs two cycle flipping computations in a row before continuing execution.

```
#define gb_next_rand() (*gb_fptr >= 0 ? *gb_fptr- : (gb_flip_cycle(),  \
                                                      gb_flip_cycle()))
```

The modification would exploit the comma operator in C to allow for two consecutive runs of the routine prior to continuing with the flow of execution at the macro invocation site; This is possible thanks to the fact C evaluates as `void` the lhs of the comma operator and assures that the rhs will only run after the lhs has finished execution, such that the `gb_flip_cycle()` returns the last element of the stateful array after having run 110 steps of the recurrence defined in Equation (1.1). For future reference, I belive it is useful that we inspect the operations performed in the macro.

First, it considers whether the current value pointed to by `gb_fptr` is negative. This holds only for the sentinel value in the array the pointer it is aliasing, and thus serves as an indication that it's time to run `gb_flip_cycle()` (though the workings of this are, yet again, unbeknownst to me at this point.) If the value yielded happens to still be within the "acceptable" range, then the underlying value is dereferenced again prior to performing pointer arithmetic by subtracting from `gb_fptr`; Note how the decrement operator is used in its postfix form, such that only upon returning the value does the pointer's address recede back by one position in the array.

The `gb_flip_cyle()` routine is said to perform 55 iterations of Equation (1.1), aiming for these to be as high-speed as possible by requesting register storage of the pointers in use. This function's body, though, is quite the sight for sore eyes; It keeps two pointers to the array holding the n random numbers (never acting on the sentinel value at index `1`,) and performs *pointer address* comparisons to consider whether the pointer at the end of each loop iteration has hit the address of the last element in the array. The problem here is that the exit condition of the loops depends on whether the pointer involved in each one, respectively, has an address that is now "beyond" the address range of the array (i.e. has an address that is numerically larger than that of the last element of the array.) Technically, one can trust that C stack-based arrays will allocate contiguous memory and thus an address that is

numerically larger than the address of the last element in the array would be outside the safe range in which to dereference the pointer, so the check is certainly not incorrect in its logic. But this is borderline unsafe in Rust.

Then it proceeds to "reset" the `gb_fptr` pointer by making it alias element at index `54` of the stateful array. I belive it resets it to the element right before the last and not to the element before the last proper because the routine itself returns the last element in the stateful array. And then because this function is really only used inside the `gb_next_rand` macro, it's expected to keep a coherent sequence of values, such that so long as we've not hit the sentinel, we return the dereferenced `gb_fptr`, otherwise calling `gb_flip_cycle()` and getting after its call the value at the very end of the array, while resetting back `gb_fptr` for the next call to the macro to start anew.

The `gb_next_rand` macro simply computes the actual formula in Equation (1.1) with discrete values for the terms R_{n-55}, R_{n-24}. This, though, is not a built-in modulo operation with the chosen m (2^{31}), but rather a (possibly) more optimized version using a bit-wise `&` that relies on the machine using 2C bit representation for its integer primitive types.

Note that, according to the docs, the `gb_flip_cycle()` routine is to be thought as reflecting the sequence of values, in the sense that they are now considered in reverse order to their initial orderings. Still, a point is made about this not affecting the degree of perceived randomness on the returned sequence throughout calls to the `gb_next_rand` macro and subsequent "flipped cycles" upon hitting the sentinel value in the array with the `gb_fptr`.

The initialization routine `gb_init_rand()` follows a process akin to the one detailed in [1], except that apparently the summary that it references bases its generator off of the assumption that only the low-order bits of the initial values (those variables allocated at the start of the routine) are the ones with pseudodeterministic significance. Then for the initial number sequence dispersion, it makes use of coprime numbers 21 and 55 because further increments expressed in terms of a modulo such as $21 \bmod 55$ allow for the iteration-based values in use with the initialization of the seed to be numbers part of the Fibonacci sequence (this is commented to be an alternative method of improvement once the seed value has determined a starting point in the precomputed congruential series.) The reason why this is any relevant for the purposes of a random number generator are discussed in [5].

The reason why the resulting C programs from running `ctangle` on the CWEB sources make abundant use of the `#line` directive is due to the fact literate programming, as conceived by D. E. Knuth in WEB, may clip parts of a given routine or general language construct, for the sake of documenting an isolated "region" of it. This in turn forces `ctangle` to parse and require a restructuring that may not be desired when debugging and using compiler-defined symbols when the C source file changes

the ordering of such lines to the one expected by a compiler toolchain. This in turn implies D. E. Knuth expects CWEB programs to be perused in their `.w` forms, and not as standalone C programs, including debugging.

Back to the initialization routine, this process mostly consists of three separate steps: (1) Assigning to each value of the stateful array a different "random" value, (2) computing the next set of values that will be assigned to such elements of the array, and, (3) "warming up" the values finally set in the array by calling for 275 steps of the array value-reflecting routine within the cycling function. The reason behind the warmup cycles being run after the example routine in [1, Sec. 3.6, p. 184] is due to the fact that the least 10 significant bits (the low-order bits we spoke of before) present a fairly predictable pattern no matter which first random number we compute. Of course, the pattern is only obvious when purposefully considering the bits of the numbers, even if small fluctuations may happen between the 9th bit and the 1st bit. The quick cycling (array member reflection) added to the initialization routine for this pattern is meant to quickly disperse values, as otherwise the first few hundred runs would very much follow step no matter the environment execution conditions.

Beyond this, there's nothing else to the generator routines. The only other function present in the public interface of the library is one for computing a uniform, *bounded* distribution of integers. As per S. Skiena, the function R_n already produces such bounded distribution, where the range is denoted as $[0, m)$, so it's quite possible the function presented in D. E. Knuth's generator is not a linear congruential engine, but can be coerced into the ranges produced by one. The reason why the routine is provided instead of simply bounding the generated number by a modulo operation is attributed to the fact that such operation would yield values smaller than or equal to $m/2$, on $2/3$ of the runs. This function should apparently (though no further elaboration on the reason why is given) try to clamp the generated value down to the specified range, while not consuming any more than 2 random numbers in the series (through the `gb_next_rand` macro.)

This seems to work especially well for values of m larger than 2^{16}, where the trend for smaller values is seen more often. Because the random numbers generated within the hot loop of the routine are compared with the largest representable 32-bit *unsigned* integer modulo m, this should technically perform a form of await operation that would evaluate the generated random number prior to itself being returned modulo m. This may sound like more than two runs (and thus more than two macro invocations to `gb_next_rand`) would be required, but of note is that the generated number is itself not yet bound to a range smaller than the provided m, even if the heuristic used to compute a number anew is based on a value bound within m (more specifically, $2^{31} \bmod m$.) It is only once the computed value yields a (possibly) larger value than those generated under $m/2$ that the routine returns the

"clamped" value $n \bmod m$ (I use n instead of the variable name `r` in the source code, for the purpose of generality.)

Pending: getting information on some omitted technicalities on the methods used for both engine initialization and number generation (though these should be found in [1].)

1.1.2 Graph routines (`gb_graph.w`)

All declarations until line 110 of the generated C code are part of the type system used for graph abstractions. It also seems to include a `verbose` and `panic_code` variables to consider for (1) possible additional verbosity in the output of graph routines (akin to current-day `-verbose` CLI options), and, (2) a custom, `errno`-like, global return code living as a static for the purposes of reporting a wide assortment of failure cases, respectively.

[3, Sec. 1.3] mentions that the generative routine for books accepts two levels of verbosity, but this module (`gb_graph.w`) only refers to the existence of a single level of verbosity, controlled through the public static global `verbose`. It may be that the actual verbosity is controlled by setting the variable to a larger or smaller value, or it may be that each module requiring of some level of verbosity implements its own flag on top of that flag. I am inclined to believe it is the former approach that the generative routines follow, as the variable itself is declared as a 32-bit signed integer, which leads me to believe that the routines extend the "accepted" range of verbosity on a case-by-case basis.

The error codes returned use a sligthly primitive approach based on the same `E*` error codes present in UNIX-based platforms, but really only using generic names for, say, a syntax error, where there may be different variations in the core source, but such subtleties are left to the engineer to find out by perusing the implementation in search of the offset added to the generic error code. D. E. Knuth allows for the following ranges to delimit the set of values that may be stored in the `panic_code` global.

Code `-1`

Some memory limit was hit during some prior operation. This is not further explained, and I don't know how would a single error buffer hold an indication of a "previous" error if determining that the error itself took place before some other error requires having had another error, and thus having had to *discard* the "previous" error to use this integer buffer for the new error code.

Range `1-9`

Some memory limit has been hit during the invocation of a C standard library function.

Range `10-19` (`10` with an additional offset `1-9`)

This denotes some error took place at the "start" of one of the data files sourcing the information that gets parsed onto the graphs. Note that we speak of a `10` *with an additional offset* because

the next item denotes variations on the errors when the error has taken place at some point in the second middle of the `.dat` file at fault.

Range `11-19` (`11` with an additional offset `1-8`)
Much in the same way as the above range, it reports some error has taken place at some point in the "lower end" of a file (the second half of it.)

Range `20-29`
There was a syntax error while reading a `.dat` file. I assume this error implies the above two ranges relate to memory corruption errors in the I/O routines, and not so much during file parsing.

Range `30-31`
There was an error in the call to a GraphBase kernel routine. This range is reserved for fairly "well-behaved," but wrong values that may still make some sense in the context in which they were issued.

Range `40-49`
Ibid., except the passed values are completely wrong and it could very well be that the overarching logic involved in the call to the reporting subroutine is also wrong. Basically, you're *stupid* (in the words of D. E. Knuth himself.)

Range `50-59`
The graph parameter is `NULL`, which is no good but especially so in this codebase, where according to the `README`, D. E. Knuth *assumed* that this symbol was defined universally as `0` and thus is used for expansions where one would naturally expect the number `0` used. **Pay attention to this item, both because in Rust this could very well be a no-op, and because the code could be quite wrong in its use of `NULL`.**

Range `60-89`
A parameter is technically correct, but it likely breaks some invariant the routine must upkeep as part of its pre/post-conditions (maybe, but this is not for sure and further inspection of the routines calling any of these graph primitives is required.)

Range `90`-onwards
This is not meant to happen, but in the name of safety (the irony,) D. E. Knuth still considers these edge cases, and binds values to be interpreted as execution paths aking to those ending at an `unreachable!()` macro invocation in Rust.

The docs move on now to the DSs in use for graph abstractions. Because of the presence of multiple types of graphs across the actual library routines, graphs use an overarching structure `Graph`, which

itself is accompanied by types for edges and vertices. The edges use an `Arc` type, while the vertices use a `Vertex` type.

Each graph uses an array to store `Vertex` structures. Then it uses a linked list to keep a record of the edges, such that each element of the linked list is an `Arc` structure. The linked list isn't anything fancy; A set of pointers in pre-ANSI C with a field `next` pointing to next element of the linked DS. In terms of standard graph data structures, this follows the same strategy as an adjacency list-based graph, where each `Vertex` contains a field pointing to the first element of its (singly) linked list of arcs (edges for undirected graphs, thought further elaboration is required there as GraphBase has some "special" treatment of undirected graphs.)

Another core tenant of the architectural design is that each one of the `Arc`, `Vertex` and `Graph` types is provided with an assortment of additional fields of a union type `util`. This union contains one of (1) a pointer to another `Graph`, (2) a pointer to another `Vertex`, (3) a pointer to another `Arc`, (4) a string literal as a `char *`, or, (5) a 32-bit signed integer. This union is said to be provided in the spirit of allowing flexibility for the algorithms using these set of DSs. This is going to get refactored in the Rust rewrite into compile-time codegen with attribute-like macros on top of some type deriving the behavior of a specific type of graph.

The set of fields in the `Vertex` structure is fairly straightforward. It associates a string literal with the `Vertex` in question, as well as a standard C implementation of a singly linked list for the adjacency list, such that only a pointer to an edge `Arc` is really held per vertex. Then an invariant holds such that if the pointer is non-`NULL`, one may safely access the `Arc` and check for the `NULL`-ness of a `next` field giving step to the next edge *with respect to the original* `Vertex` *from which the pointer to* `Arc` *originated*. Each vertex is also outfit with 6 `util` unions.

The set of fields in `Arc` structures hold information about the vertex that some other vertex is connected to (implying here that such relationship only really holds true when sourcing an `Arc` from the adjacency list of a `Vertex`, where this latter vertex makes up the other end of the edge,) the next `Arc` connecting the source `Vertex` (in the same context as the above note) in such vertex's adjacency list, and a length associated with the edge (possibly for weighted graphs or otherwise embedded graphs.) Unlike `Vertex`, it holds a smaller number of `util` unions, namely 2.

This design for vertices and edges does force the same set of restrictions on graph representations as basic adjacency lists presented in [6]. A more optimized graph backend with a wider assortment of DSs allows for more flexibility, and will likely motivate the implementation of the Engine API as a Rust trait-based implementation of this and other routines.

The docs move on now to explaining the memory allocation strategy followed to try and make the system efficient. This is quite possibly going to require both considerable refactoring in Rust and benchmarks to test that whatever efficency was attained with D. E. Knuth's methods continues being *attainable* with Rust methods. The initial implementation, though, is *not* going to make use of any of the attempts at memory allocation optimizations that are present in the GraphBase codebase.

The way memory allocations and memory resources are freed is by means of an abstraction layer over basic memory handling library functions of whatever pre-ANSI C the author used. The entrypoint and main resource handle in all involved functionality is the so-called `Area` type(def). This could be said to be modeled after the `allocator` API in the corresponding module of Rust's `std`, only instead of being a complete memory allocator, it serves as a bridge to mediate between the default allocator of the C compiler used with the result of `ctangle` transpilation, and the graph routines requiring allocation (which almost always happens exclusively during grpah construction.)

The `Area` type is, at its core, a singleton array of pointers to other `Area`s (to allow for direct dereferencing through array-to-pointer decay without expecting the user to provide the explicit address-of operator in parameters to memory allocation request routines.) When rid of this level of indirection, the underlying structure reveals to be a `char *` (this is from back when `char *`s were used instead of `void *`s because this latter type wasn't valid, "generic" C code,) denoting the address at which this block of "memory area" starts. This is also accompanied by another field `next` pointing to another `Area`. This field serves the purpose of indicating which memory area was used "before," where the concept of time is defined as the memory areas whose storage was depleted, and thus the memory areas that are to be tracked for resource freeing.

The basic use of `Area`s follows that the user should consider in which segment of the program the resource handle is stored at, and if it is outside the statically initialized data segment, so either in *bss* or in stack space (I don't think it a good idea to have it in the free store) then it should proceed to call the `init_area` macro to initialize the array to `NULL` (this, for one, is a correct use of the symbol.) Each `Area` represents a fixed-size heap allocation, and each call to the `gb_alloc()` routine with a block size and an `Area` passed as parameters is expected to either (1) allocate on a `NULL` memory area, (2) extend a non-`NULL` memory area, or, (3) return `NULL` (as a status code, so quite wrongly assuming it expands to `0` on all platforms) if either `Area` cannot allocate any further due to system concerns, or due to a request size surpassing `0xFFFF00` bytes.

If such an error takes place, another global (static) status code variable, `gb_trouble_code`, is set to a non-zero value such that if performing multiple allocations in a row, an error on any one of these may be detected by means of a conditional check on said status code.

A convenience macro, `gb_typed_alloc`, is provided to allocate the required amount for a given type, through a quick `sizeof` on said type and a multiplication by the requested amount of objects n of that type. The resulting value is passed as part of an allocation request to `gb_alloc()` representing the number of bytes. Because this last routine requires an `Area` to be passed as well, it both allocates the required heap memory on said memory area, and returns a pointer that is cast to a pointer to the type passed as part of the `gb_typed_alloc` macro invocation.

The `gb_alloc()` routine, at its core, only really `calloc()`s 2^8 items, each made out of `n` bytes passed as a parameter (having applied the ceiling function to those `n` bytes such that they are a multiple of the platform's pointer size, `char *` back when the program was written instead of `void *`, but 8 bytes either way (LP or LLP memory models would do just fine, and the program likely predates the time when proposals over these two "ended" other memory models.)) The specific size of each of the elements to be allocated is not completely clear to me just yet; D. E. Knuth computes ${}^{n}/_{m} + {}^{2m}/_{m} + {}^{m-1}/_{m}$, which should theoretically resolve to $\approx {}^{n-1}/_{m^2} + {}^{1}/_{m} + 3$. But the meaning of this is lost on me right now. Maybe it only serves as a range restriction as per the notice in the documentation, which comments on old-style C having a hard limit on the byte size passed as the first parameter to `calloc()` (I've not found any such warnings on current-day, BSD-derivative manpages.)

If the allocation is sucessful, the returned region of heap memory goes through three main "manipulation" steps, in the order described here.

1. The address lying `n` bytes (post-ceiling function) forward from the returned `calloc`ation is cast into the element of an `Area` (the underlying `struct area_pointers *`.)
2. A separate, temporary `Area`, is dereferenced (making use of array-to-pointer decay to reach directly for the first element, again a `struct area_pointers *`) to have assigned to its (pointee's) `first` field the starting address of the original `calloc`ation, and to its (pointee's) `next` field the dereferenced `Area` that was originally passed to the `gb_alloc()` routine (ibid.)
3. This latter (original) `Area` is again dereferenced to have its single element pointer `struct area_pointers *` alias the dereferenced element pointer of the former (temporary) `Area`.

I don't completely understand the n size computation, but that matters not for the refactor.

The final state of the original, parameterized, `Area` has its single pointer element hold `first` to the actual start of the `calloc`ation, and `next` to `NULL` on the very first call to a new `Area`, and to the previous (depleted) `Area` (which itself was the `Area` pointed to by the passed parameter of `gb_alloc()` prior to this routine realizing the size limit for a single `Area` was hit.) Following, I elaborate further on the inner workings of `gb_alloc()` during the second and future calls.

Tracing the behavior of the `gb_alloc()` routine across a second call would have the passed `Area` be already allocated with a reference to `NULL` in its only element's `next` field, and a pointer to this same element's own adress minus an offset equivalent to the allocation of the current block in its `first` field. Fast forward to the end of the memory allocation request routine, and once the `callocation` has yield a pointer, the heap memory towards which the pointer leads is cast into the memory area's underlying pointee element type (`struct area_pointers`,) prior to performing an assignment to the `first` field of the temporary `Area` within the routine corresponding to the new block, after which the `next` field will be a pointer to the previous `Area` passed to the function, so that this same memory area holds a pointer to the start of the new allocation in its signle element's pointee `first` field, and another pointer to the allocation we had before in its `next` field.

The only way this represents a win in terms of efficency is if `calloc` is trusted to return contiguous memory allocations from some source array on the free store. This scheme would then have each `Area` hold both its own allocated size request (if it doesn't surpass `0xFFFF00` ≈ 16 M bytes,) and a pointer to the start of the previously allocated `Area`, which if the calls to the C standard library functions work as assumed by D. E. Knuth, should yield contiguous addresses after $255 \times n$ bytes.

This is not the case in modern systems, and has never been the case in both ANSI C. The win in efficency is arguable. Having single-threaded arena-like behavior encapsulated in a linked list-like `Area` object is not any better than having an array of pointers to memory allocated on the heap through any of the `*alloc()` routines in standard C. Assumming they are all just resource handles acting as a bridge between the default system allocator and the library user, it's not feasible to try replicating this same strategy in Rust (because even if it relies on initialized, non-null memory ranges, it's still type-unaligned memory that would require calls to `std::mem::transmute()` to force a bit-level pattern reinterpretation, which is no better than `reinterpret_cast<>()` in C++, and that's dangerous in and of itself.)

A potential implementation that would both (1) assure *contiguousness* in the allocated memory, as well as, (2) allow for modern-day heap allocations without linked list behavior would be to use memory mappings from the UNIX API. This would be limitting for non-UNIX API users, but could be both safely implemented with the `rustix` safe wrapper around these syscalls, and is also how `malloc()` is implemented starting at certain memory request sizes in glibc. More specifically, this would use private, anonymous mappings on the process' virtual memory address space, letting the kernel allocate memory wherever it deems safe, while keeping a very similar memory arrangement, in terms of a pointer leading to the next `mmap`ed memory region. A single resource handle would hold reign over a conservative `Vec` capacity-like `MaybeUninit` region of memory, and any requests through the corresponding library API woudl be forwarded to such handle, which would return a

pointer to the address range with the specified amount of bytes in the request, but would fail if the references taking up space in such region have depleted the required memory to fulfill the latest request. This approach, though, presents three issues.

- The `mmap`ped memory region must be large enough while still being conservative on the available memory provided to the process (must account for OOM in Linux and most BSDs.) Chapter 9 of the nomicon can likely give some clues on how to better solve this issue, where the reference implementation for `Vec` is given along with an explanation on correctly doing OBRM.
- The resource handle would be forced to keep a reference count of both the referees making each request, as well as of the memory regions that they themselves hold resources over. This is pretty much replicating garbage-collected behavior of some languages, except the reference count would only go down if a corresponding call to the deallocation routine is performed by the requestee that initially made the resource allocation call. This can be modeled through the `Drop` trait and associated `drop()` function on whichever generic type represents the answer to the memory request, but it doesn't seem simple to further expand for other types without dropping to `unsafe` and using `mem::transmute()`.
- Each call to `mmap()` would not be guaranteed to return a contiguous memory region, especially not if the starting address is kernel-dependent. A possibility would be for the initial `mmap()` call to be the *only* call to this syscall that lets the kernel pick an arbitrary address, such that subsequent allocations rely on having a handle over both the starting address and the offset that it is bound by, then performing overlapping `mmap()` calls with the first parameter of the syscall denoting the starting address of the new range, and the parameter denoting the mapped size being a bounds-checked range over the original call's range. This could still potentially require another call to `mmap()` to fetch another chunk of memory from the process' virtual adress space, but if fairly decent heuristics can be found to set an initial "good" size, it should not happen as often as one would expect. This would fit the bill quite well with the allocation-heavy behavior of graph construction routines, as those are the ones that most often require of new heap memory.

I leave these ideas in the backburner, but it's quite likely this specially isn't getting into `0.1`.

There's also a function to quickly deallocate the `Areas` that some global static handle kept track of through the linked `next` fields of the underlying type, but that's irrelevant for the Rust rewrite.

The docs on the `Graph` type explain the existence of an assortment of routines to both create a graph, and attempt to efficently handle both vertices and edges. Most of the strategies followed by D. E. Knuth are completely useless nowadays, and would require the use of platform-dependent code to rely on pointer arithmetic behavior that is not predictable outside `MIPS`, `x86-32` and `x86_64`. A consequence

is that the type system will be the only thing ported over to Rust, as the memory allocation practices are, in general, not apt for a portable, non-UNIX dependent program.

The type at hand considers 5 fields of chief importance for the graph, two `Area`s for data on arcs, strings and other auxiliary information; And 6 `util` union types as well as an additional field denoting a string whose single character sequences provide meaning to each of the utility fields.

The first five fields include a heap-allocated array of `Vertex` type (so linked pointers,) the number of vertices as well as the number of arcs in the graph, and two `Area`s for main storage allocation of `Arc`s and `Vertex`s, as well as a auxiliary storage allocation for some generative routines.

The `util` unions on the graph are very much akin to those found in `Vertex` structures except that the `Graph` type also holds a discriminant character array indicating the purpose of each of those utility fields, both for itself and for the vertices and arcs within it. This is truly a showcase of the severe limitations and safety issues with C-style unions, and likely also the reason why the Rust rewrite will not follow that approach. As a consequence, even though it's a string field to C compilers, this is in actuality a purely single-character based array where each element denotes, in uppercase ASCII alphabetic symbols, the purpose of the utility field at any given time (this information is stored also for the purposes of exporting graphs with the `gb_save` module, because these exported files embed graph information and need to be lossless.) The alphabetic symbols in question follow the same semantics as the names in the union declaration. For each of the possible fields, it considers the complete set of union fields (including the fields on the `Graph`, and those on the `Vertex`s and `Arc`s contained within it,) adding the `Z` character to indicate that the field in question is not in use.

The distribution for this single-character array is as follows.

Range `0-5`
: They denote the meaning of `util` fields in the contained `Vertex`.

Range `6-7`
: They provide significance to the `util` fields of `Arc`s. Recall this type only had two such fields.

Range `8-13`
: They provide meaning to the `util` fields in the overarching `Graph` type.

`Graph`s also holds an `id` field that is not meant as a UID, but rather as a form of identification against the routine that invoked creation of the graph. this is because most graphs are created after a generative routine has been issued with specific parameters, as explained throughout [3, Ch. 1].

Because these fields' main purpose is that of providing a discriminant for the union and because D. E. Knuth expects their use to be most often found in I/O routines, they could be completely replaced

with a trait-based implementation on the formatter API such that exporting was modeled after `serde` serialization/deserialization, which could make for an API that would be as extensible as the users would require. A possible implementation would go through using an attribute-like macro that would let the user choose a serialization method of choice by using the inherent type consumption of these macros; Given some annotated type, they could produce the same type with the corresponding `serde` macros applied. This may or may not be feasible, depending on whether the Rust parser performs another pass through the AST of a source file after evaluation of an attribute-like macro. **I leave this idea in the backburner.**

The graph creation routine, `gb_new_graph()`, performs two main operations: (1) allocating space for the graph and a parameterized amount of edges n passed as part of the routine, plus an additional amount of vertices due to some algorithms requiring so, and, (2) setting up the value of a few file statics that cache part of the state of a graph upon creation to apparently make more efficient the use of certain routines often called right after the creation of a graph.

During initialization, the above routine will also set the `util` union fields to hold the discriminant variant standing for no information, namely character `Z`. This function will also set up the associated string identifier of the graph (the so-called `id`) to non-UID that is meant to be either immediately changed through one of two other routines, or left as part of the auxiliary graph IDs to be used in these two latter routines.

The functions concerning themselves with graph ID-setting provide either a single graph-to-graph way to perform such ID-setting operation, or alternatively a 2-graph to single graph ID setting routine to set a graph's ID from the ID of two other graphs and some additional strings. These routines will, for now, not be documented as a better approach would be the implementation of a UID algorithm for the graphs (which should be fairly simple, considering the `id` field serves as a means of inter-graph communication in the generative routines, and thus has no significance at the cryptographic level.)

As previously mentioned on the graph creation routine, one of the two main tasks it performs is to set up some external globals to cache graph state, and make sure fallible routines can indicate failure beyond the global error status codes (both the one set with the allocation request routines and the one found in the routines concerned with random number generation.) The set variables include symmetric equivalents for (1) arc allocation, as `Arc`s are allocated in bulk with `Area`s in `gb_alloc()` and a pointer is required to indicate both the availability of an arc that has not yet been added to the graph on a logical level and the possible failure state that should be compared with its symmetric counterpart, (2) string allocation, ibid. considering the use of an `Area` specifically for the purposes of storing both this and the prior resource, and, (3) graph allocation, ibid.

The routines involved with arc/edge creation are themselves wrappers for a more primitive routine that uses the current graph's main `Area` (recall the existence of two `Area`s, one of which was used for auxiliary purposes) to allocate a conservative default of 102 new arcs, irrespective of the amount actually required at any given point throughout program execution. There's two wrappers because one covers the usecase of having single-directionall arcs for directed graphs, while the other considers bidirectional arcs to denote the existence of an edge in an undirected graph (more generally, if a graph is undirected, it is also a multigraph where any two vertices given by (i, j), always have two arcs $i \to j, j \to i$ of same weight, which voids direction of any meaning, and thus makes the graph *undirected*.)

Note this routine also uses the previously mentioned file statics to indicate either that the next available arc is the one offset by one byte after the address of the returned `gb_allocated` amount of arcs (thus one past the address denoted by the `first` field of the pointee of the element of the main `Area` of a `Graph`,) or that the allocation failed, and the "bad arc" variable must be assumed to be pointing past all elements that *should* have been returned from the `gb_tpyed_alloc` macro invocation.

The core logic of the routine, though, is fairly simple. If the global pointer holding the address of the next available (but not yet part of the graph) arc indicates it has reached the global holding the address found one element past the end of the `gb_allocated` `Area` currently tracked by the graph's main `Area`, then it's time to call `gb_alloc()` again and request another 102 arcs in a single block of memory that will now make the current graph's main `Area` become the holder of such resources, while keeping the previously allocated `Area` (used up) in its single element's pointee's `next` field (yes, the *previous* memory area is held in the `next` field.) Otherwise, it simply advances the global pointer tracking the next available arc in the current `Area` by 1 and returns another pointer aliasing that same pointer prior to having advanced it (thus the returned pointer is the one that was available on entry, while on exit, the tracking global pointer is past one element from the one returned and indicates again the next available but not set arc.)

The invariant held over which one is the "current" graph, and thus which `Area` should be the one being manipulated is weak in its semantics. Among the globals participating in the above routine, there's one that also tracks the graph that was created last with `gb_new_graph()`. That's the one holding all the responsibility with respect to which graph's `Area` is to be manipulated, pointed to, invalidated or extended. In this routine, a pointer is set to alias the returned graph. This has some severe limitations, that are only partially (and wrongly) bypassed by using the `gb_switch_graph()` routine to temporarily use some of the union fields in the "current" graph to store the information of such graph, and reset all union fields of the graph that is now meant to become the "current" graph. Then one may operate with the global referring to the "current" graph knowing as well that this refers to the graph passed as a parameter to the graph switching routine.

This procedure forces the "current" (prior) graph's union fields to be invalidated, and thus expects the user to only use those fields after any logic involving arcs and edges is completed. Otherwise, the union fields themselves will not restore their previous state. The most noticeable limitation, though, is the requirement on the passed graph having to have already gone through another graph switching routine itself to be "switched out." Because initially no graph could possibly have been switched out, D. E. Knuth offers as an alternative to pass `NULL` to this routine right after the creation of a graph that is "planned to be switched out", only for the side effects it has on the global, which are the ones that force the requirement of the "current" graph (denoted by the corresponding global) having had to be switched in the first place.

This is going to need severe refactoring in Rust.

The wrapper to add an arc to a directed graph performs no logic beyond changing pointees of graph, vertex, and arc pointers, such that the new state of the adjacency list with an additional edge remains senseful. The wrapper around edge creation in undirected graphs is slightly more complex, but its implementation design will likely have to be trashed in Rust, as it heavily relies on the numerical value of pointers by directly comparing their adresses to determine a set of invariants over whether an arc can be found nearest in memory to its inverse arc (where the mapping of inverse arcs is that in which both such arcs, the *un*inverted and the inverted, make up the concept of edge, devoid of direction.) This also forces further restrictions on the call chain of arc creation routines as apparently mixing up this one with the other wrapper or with the underlying primitive function will lead to possibly undefined behavior or otherwise an ill-formed program.

No further comments will be made on these routines because they only perform trivial (and non-C standard conformant) pointer arithmetic that will be completely replaced in Rust.

The last routine the documentation comments on is concerned with string allocation for the purposes of vertex/arc labeling, making up the other "client" of the memory served by the main `Area` in a `Graph`. This function follows the same trend as the ones last commented on, using a few lines of pointer arithmetic to advance the pointer to the first character in a character array, and avoids using library functions for appending strings (`strcat` being the only one available back when this was written,) as they can be fairly inefficient. The approach to either returning the next available string or otherwise trying to allocate memory equivalent to that of the length of the string is used in the same way as with the arc allocation routines (except `gb_alloc()` is called directly with the requested size of the string because D. E. Knuth likely assumed that either only the ASCII character set would be used, or otherwise the user would be prone to manually compute the length of any of their (1) wide strings, or, (2) UTF-8 encoded strings.)

If the corresonding global pointing to the next piece of memory in the `Area` of the "current" graph (itself denoted by another risky global) does **not** have the same address as the *other* global pointing one past the end of the valid, allocated `Area` and there's enough space available in that same `Area` for the length of the string (i.e. offsetting the former global does not yet yield the latter global,) then the routine deems it safe to perform straight pointer arithmetic on the buffer starting at the in-bounds ("good") global to yield as many bytes as the requested length indicates. Otherwise, it attempts to allocate either the size of the string if this surpasses the default size request, or otherwise the default size request. Calls with a size smaller than the minimum in bytes are clamped to that minimum due to the fact requests to the allocation mediator routine provided by the library advise against using sizes smaller than 1000 bytes, as starting from that size the amount of syscalls required to actually reserve such heap memory, if available, should decrease. I highly doubt this is the case anymore nowadays, and D. E. Knuth himself doesn't provide any reference to implementations of these C library functions that would lead one to believe this has changed.

This whole routine is completely out of the Rust rewrite. Maybe refactoring the standard Rust `String` struct to rid it of unnecessary behavior would be an option, but even then, the `std::string` module knows how to be efficient. And the most of this whole memory allocation strategy that's getting into the refactor is (maybe) an abstraction layer over the `std::allocator` structs to further mediate between the library user and the system allocator (only if that's possible without forcing it on the dependent crate.)

There's also a routine to free the resources of the graph, that calls the corresponding memory-freeing routines on `Area`s, and `free()` on the passed graph (`Area`s are not used to keep a record of `Graph`s in `gb_graph.w` but that may very well be the case in the generative routines, considering D. E. Knuth exposes memory areas explicitly as part of the public interface of the library kernel modules.) This is not going to be discussed further, because it's completely useless with RAII in Rust. The only possible modification to the `drop()` trait method that would be required to implement similar semantics would require overwriting the `std::allocator` structs, which may or may not be possible if the dependent crate is also affected by such changes to the system allocation Rust API. The `std::alloc` module does mention that the attribute `[global_allocator]` can only be used once on a crate, but doesn't further specify whether dependencies of that crate will be forced into using the same memory allocator. It does mention, though, that recursive dependencies of a crate (so I'm assumming this includes both explicit cargo dependencies and whichever dependencies these themselves include) can only ever specify this attribute once. This does mean that if this library (GraphBase) includes code that overwrites the allocator, (direclty or indirectly) dependent crates will be forced into using the same allocator, as that's what I can infer from the fact that a no two crates involved in a package can have the attribute used more than once.

Initially, I believe it best to implement the library in terms of `std` defaults, and worry later about how could the D. E. Knuth memory management strategy be implemented in Rust.

The docs now move on to the part of the library implementing functionality for fast O(1) vertex lookup through their string labels. This is considered in the context of hashing with separate chaining, using the derived results from D. E. Knuth's conclusions on the number of probes that would be required to compute the number of comparisons between the (hashed) input key and some search key that would change as the symbol table was traversed. In the initial implementation, I believe the Rust code should just use the algorithms in the standard library to compute the hashes of the string keys, and store them in either one of an auxiliary hashmap stored within the overarching graph representation/DS, or otherwise use an extension interface to allow the users to make arbitrary use of the hashmap DS as they see fit. A possible implementation for this would be the use of proc-macros for compile-time addition of a field in the graph DS, such that use of that feature is gated to a user request on codegen.

Another reason to get the proposed strategy replaced is due to the fact the number of nodes after creation of the hashmap ought be kept consistent by the user, as otherwise the whole hashmap would have to be rehashed. I'm not completely sure about the extent of this limitation in Rust's `HashMap`, but at least the user is assured that if such thing were required, it's quite possible that the container could make do by itself without manual intervention. There's also the fact that auxiliary global pointers are messed with, which is not exaclty ideal. Add to that the use of `util` unions on the `Graph`, and we're pretty much binding the user to another set of strict requirements on the extent of their use of such fields; Especially considering the note warning against possible system crashes if one does not meddle with great care.

In Rust, I think both the union fields and the additional hashmap could be implemented in terms of proc-macros. These would allow the crate user to generate, on a case-by-case basis, instances of the `Graph` DS proposed by D. E. Knuth with well-defined extensions to the `structs` through attribute-like macros. The feasibility of this, though, is quite uncertain. On the one hand, tagging existing structures with attribute-like proc-macros seems like an option worth considering. On the other hand, one soon realizes that such need would not arise in a dependent crate unless the user designed themselves the graph DS. Then, if one considers providing pre-existing data types for graphs, the attribute-like macros would be forced to either act as extensions to all generated instances of the graph DS, or otherwise… God knows. The ideal behavior, just to get the idea out, would be for the user to create the graph DS and then allow them to, on a case-by-case basis, tag some given instance with the attribute. Such thing would allow the underlying proc-macro to create a new type for the graph (with a possibly mangled name that would include an custom identifier; Maybe a hash key relating

to the identifier of the variable with the attribute?) such that this type could be used for the purposes of plugging a given struct into a generic interface expecting a type that implements some trait that the newly generated structure could also implement. Whether this is at all possible in any context in which a user is required of a type implementing a given trait for a specific function, is very much related to the extent with which the Rust grammar allows flexibility in the declaration of new types and the scopes in which this can happen.

The proc-macros idea seems feasible. The only thing to consider as a notable difference from what I commented on above is the way one would encode information relative to a newly generated type. Simply using a hash function is not enough, as that forces holding the invariant that provided two equal input keys, the same hash is returned. Of course, if we consider variable shadowing, this stops providing the proc-macro with unique IDs. The solution would be to either (1) consider a UID generator (through an external library or through a manual implementation of one such algorithm,) or, (2) consider hashing in terms of the `Span` associated with some token in the `TokenStream` passed to the underlying proc-macro function signature.

This seems very much possible. Apparently, only *items* in the Rust grammar can be considered as being capable of having an outer attribute applied to them, but it turns out statements, including `let` statements, may also have outer attributes applied to them. The extent with which some such outer attribute may have an effect on a statement, as per the reference's concepts of both, is not yet clear to me, though. The reference speaks of "applicability" when referring to the possibility of having outer attributes be used on a statement, including a `let` statement, but such "applicability" is not further developed in the context of whether it is restricting in nature or if it only offers an example of the uses it currently finds in the `rustc` compiler.

In the case that `let` statements also applied here, then the afore mentioned idea could very well be feasible. Regardless of whether the type inference algorithm was run prior to forwarding the `TokenStream` to the proc-macro, it would still be possible for the proc-macro to attempt its own form of inference on the raw syntax.

The proc-macro implementation, though, would be best served by an outer attribute that applied to some tuple-like unit struct such that the user decided on the token identifier and scope of the type about to be generated. Because the attribute would likely have to be the same for the purpose of accessibility to the user if the API ever extended beyond this basic functionality, this should prove to be enough. Because outer attributes (as attribute-like proc-macros are, when parsed post-tokenization) are expected to completely consume the `TokenStream` of the second parameter to the macro's underlying function signature, taking in the user type and generating all of the fields, as well

as the required trait implementations for some generic function expecting a type implementing a certain trait, should make this whole idea possible.

Of course, further refinements need to be made to actually come up with a macro that doesn't try to cover too much ground. This would likely mean considering whether it's idiomatic to expect the user to have some trait automatically implemented from simply invoking an attribute-like macro, and not from having a traditional derive macro implemented on the resulting type.

I'm going to leave this in the backburner while I continue inspecting the GraphBase codebase.

1.1.3 I/O routines (`gb_io.w`)

This file contains all of the logic concerning the processing of input data files within GraphBase in case a user is in possesion of some file exported through the (non-kernel module) `gb_save.w`. This, indeed, implies that the file does not exist as a set of both input *and* output routines, but rather as a set of routines to be used exclusively for both checking the contents of a file and getting its contents parsed into structures that GraphBase understands (though I'm not so sure on the latter.)

These routines seem to provide functionality mostly specific to either the part of the GraphBase not implementing functionality directly interfacing with the core logic of the program, or otherwise interfacing with both input and output in the `gb_save.w` module. The only thing requiring a reimplementation in Rust is going to be the backwards-compatible interface to allow reading in files through the same "protocol"/"language" as the one D. E. Knuth uses in the origina GraphBase. Even though I plan on allowing the user to automatically derive the internal types used for the whole codebase with an arbitrary serialization format through the `serde` crate, it's very much a necessity to have all `.dat` files still work with the rewrite.

This would force the implementation of at least part of the non-parsing-specific routines in the core module. Things like the universal character set that D. E. Knuth uses instead of restricting the symbols on the data files to either one of (1) ASCII, or, (2) ECBDIC (or possibly another, completely different, post-90s character set encoding) could be replaced simply with Rust's native UTF-8 strings. Other elements of the interface explictily exposed to the library user should straight up be removed, considering a large part of the prep work prior to parsing the data files is tied to the limitations of C and computers as a whole back when the GraphBase was originally written.

The system-independent encoding that D. E. Knuth uses is not completely system-independent, because the setup routine of the `icode` array holding the numerical values to which some character maps is performed through a function that expects the user's machine to evaluate the numerical value of each one of the characters that the `imap` string (really only a string out of convenience, as each single-byte character is layed out as if they were contiguous elements in an array) contains. Such

a value is assured to change in each non-ASCII or UTF-8-compliant system, and thus the actual offset added to the start of the `icode` array while filling it with the accompanying numerical value is different depending on the C runtime's evaluation of the character, which is most likely also tied to the character set encoding of the system in which the program is being executed.

Because current-day limitations on character encodings do not radiate from this type of issues, I believe it best compatibility is only upkept with the parsing logic.

After having read all of the docs on the parsing logic, it should be fairly simple to rewrite. The only parts of a `.dat` file that GraphBase expects to be conformant with a specific grammar are the first four lines and the last line of the file. Every line of a `.dat` file should contain at most 79 characters, out of which the first line needs to contain the first few characters as a substring matching the following regex.

```
^\* File "([^"]+)".+$
```

The second and third lines of the file are only matched against the same `*` character, and can thus and often will (at least in the GrahpBase repo data files) contain some description of the source/purpose of the data set, as well some licensing on how would the author like others to distribute the file.

The fourth line of the file will include information relative to the checksum, which allows computing the "expected" contents of the input string, but requires reading in the entire buffer prior to determining if the chekcsum formula adjusts with the expected checksum read in this line. The details of the checksum used by D. E. Knuth will be commented on later. The following regex describes the expected line formatting.

```
^\* \(Checksum parameters (\d+),(\d+)\)$
```

The first capture group in the regex corresponds with the expected number of lines after the checksum (fourth) line, *not* counting the last line (the one with no data, but special formatting, where we consider as *special formatting* any lines starting with the `*` symbol.) The second capture group corresponds with the final "magic number" that D. E. Knuth uses to compute the checksum from the entire contents of the file, through the formula we'll discuss after we're done with the grammar describing file formatting.

After this line, the parsing routine simply sets up the required (global, and thus inherently unsafe) buffers for other routines to manually continue the parsing process on the rest of the file contents. Each of the lexer routines checks first if the start of the line contains the `*` symbol, and if that's the case, it passes control over to the function that checks if the total number of lines read thus far adjusts itself with the expected amount read in the fourth line (the first capture group in the regex.)

Once the specified number of lines has been read through other routines (that are quite possibly going to get replaced in Rust,) another parser routine is expected to be called to check for the contents of the last line in the file to conform with the following regex.

```
^\* End of file "([^"]+)"$
```

We move now onto the checksum formula that D. E. Knuth uses to check that the contents of the file match with the second parameter of the fourth line, itself corresponding with the resulting magic file that the library routines should have gathered from computing the output-dependent series.

The checksum is computed in terms of the formula

$$\left(\sum_{l} 2^{l} \cdot c_{l}\right) \bmod p, \tag{1.2}$$

where p is a large prime, and c_l depends on D. E. Knuth's character set encoding.

Each possible value of c_l corresponds with a numerical value that maps 96 admitted characters into a symbol table that hashes them into the range $[0, 96]$. The checksum is then computed by reading the characters from each line of the file and getting, on a character-by-character basis, the hashed numerical value added to some initial value a, itself starting as the old value of the checksum or as 0 when reading in the first line of input, in the following loop formula.

$$a = (2 \cdot a + c_l) \bmod p. \tag{1.3}$$

This recurrence relation should hold so long as the string yields a non-null character, which is to say until the passed string standing in for the current internal cursor position in the open file descriptor hasn't hit the null terminator at the end of a line. Each "old checksum" (the a in the above formula) is then added to the value of the newly computed temporary (upon hitting end of line by hitting the null terminator of the passed string) for the routine to return the new value of the checksum as a function of both. This is supposed to evaluate to Equation (1.2) once the entirety of the file has been read (where we define *entirety* as all lines post the checksum-parameterized (fourth) line, and prior to the last `*`-prefixed line, indicating the end of the input data set with the same name as included in the capture group of the first `*`-prefixed line regex, for a GraphBase-conforming data set.) The result of the last computed checksum is the one that is then compared with the second parameter of the fourth line of the `.dat` file.

GraphBase makes no context-free grammar specfication beyond providing library users three lexer routines to either (1) read in a string until meeting some other passed delimitter, (2) read in a digit in some given radix d, by checking through the `icode` array the numerical value of the read number;

This is possible because all character-encoded numbers are meant to map to the same numerical values in D. E. Knuth's own encoding, or, (3) read in a whole number by performing an operation akin to that of parsing a single digit, but instead looping and adding up the values to some temporary k that is returned with the correct powers of the radix passed to the function for each digit of the processed number. This implies that the actual data between the first four lines of the file and the last line is only expected to comply with the conditions on line length (79 characters, not accounting for newline termination,) on system-independent character set use (96 characters including the standard 94 visual ASCII characters, the `\n` escape sequence and the whitespace separator,) and on the starting character in the line (can't use the `*` symbol, as that denotes the end of the input and the data set would fail to hash correctly.)

The notes on the routines for I/O should be mostly done now. The rest of the work left on the kernel routines concerns itself only with the sorting module, and actually understanding the random number generator now that I have possession of volumes 2 and 3 of D. E. Knuth's magnum opus.

1.1.4 Sorting routines (`gb_sort.w`)

This module holds the routines and types used to perform linked list sorting of any type involved in the GraphBase graph primitives (so any type among `Graph`, `Vertex` or `Arc`.) The nature of these subroutines is not one where sorting is performed along with some other type-specific operation, but rather one where the elements being sorted are themselves abstracted as pointers to `nodes` such that sorting is done independent of both (1) the pointee's type, and, (2) of the actual type used for these `nodes`.

The reason then for using a specific type (`node`) owes to the fact the sorting algorithm requires of the properties of a linked structure, whereby one instance of such a *node* is always bound to yield the "next" element in a *list* collecting all elements under consideration for sorting purposes. But the overarching container with pointers to each of these elements isn't required to hold a specific type of pointer; Instead, it considers pointers (of pointee `char` type, because `void` wasn't legal C back when GraphBase was written) to implicitly denote to the library user that for the (sorting) routines to work correctly the minimum "interface" for the sorted-through elements is supposed to have as its first two fields the same two fields as those in the `node` type, these being the only ones used in the GraphBase sorting routines.

In terms of the actual sorting algorithm, it's radix sort with radix 256, and 6 digits under consideration. The details of this algorithm in conjunction with the linear congruential engine used for random number generation (under `gb_rand.w`, and commented on in Section 1.1.1) are still something I'm not completely confident I understand.

I will proceed first with the logic involved in the form of radix sort used by D. E. Knuth. The algorithm, as explained in [5], considers a set of records n that is equal to the total amount of numbers to be sorted over. This abstraction is necessary because each of those records keeps both a `key` field, as well as a `link` field, such that in actuality it is a form of (primitve, singly) linked list that, initially, has every single element's `link` field pointing to `NULL`.

This abstraction is provided by the example `node` structure, itself having the exact same `key` and `link` fields as the ones D. E. Knuth exemplifies in [5]. Then, it keeps track of a collection of as many queues as the numerical value of the radix for each of those digits is. Following, for as many iterations as the number of digits in the largest of the `node`'s `keys` under consideration, the algorithm will consider the iteration number as the queue to operate on, and subsequently proceed to add the `nodes` whose (`key`) digit at the position currently under consideration (the iteration number) is equal to the iteration number. This introduces the second constraint imposed by these sorting routines; The structure in use (in place of the example `node`) must additionally employ 32-bit unsigned integers in its `key` field (though this is also due to the range of the random number generator used for the first two passes of radix sort.)

It will do this consistently for each of the records (the `node`'s numerical `keys`,) and then it will call another subroutine ([5, Alg. 5.2.5H]) that will link the top elements of each queue with the bottom element of the queue following that one queue, where the ordering relation between queues is upkept thanks to the contiguous collection within which the queues themselves are contained. This step allows the algorithm to repeat the above steps (the steps explained in the previous paragraph) on the next least significant digit of the numbers (so starting from the least significant digit, it moves to the right,) producing a completely different order, but this time not considering the records in the provided order, but rather in the order that they were left on (recall they are really nodes in a linked list) after performing the above subroutine to connect the top record of each queue with the bottom record of the next queue.

Repeating this for as many digits as the largest `key` in the collection being sorted has, allows a "progressive" form of implementation, bounded in the `gb_sort.w` module at 6 passes.

From looking into the implementation of radix sort in [7], it seems the above explanation applies to the method followed by D. E. Knuth in [5], but it isn't necessarily unique nor does it strike me as any better than the proposed approach in the former reference. The algorithm proposed by T. H. Cormen, C. E. Leiserson, R. R. Rivest, and C. Stein simply considers each of the digits of the multiset of numbers in the input collection, and proceeds to apply some other stable sorting algorithm only on the digit under consideration in the current iteration. This keeps repeating itself for as many times

as there are digits in the largest number in the collection, padding with left zeroes numbers with a smaller amount of digits.

The algorithm of choice as per [7] for stable sorting each digit is *counting sort*. Following, I provide a summary on its workings, based off of the explanation given by T. H. Cormen, C. E. Leiserson, R. R. Rivest, and C. Stein.

The routine involved here is not in-place so there are, at least in theory, two more memory allocations performed on each call to counting sort. The initially allocated array has as many elements as the numerical upper bound on the input array, plus 1; That is to say, it allocates memory for as many elements, plus 1, as the largest contained number in the array to be sorted. This procedure is thus best used when it is known in advance that the largest number contained in the input array is the number of elements in the array or ever so slightly larger than it.

After this, it performs an initial, linear cost pass over the input array to consider the amount of times the index of each element in the newly-allocated array repeats itself in the input array to be sorted. The new array thus serves the purpose of a frequency table akin to that used when building and operating with a Fenwick tree (also known as a Binary Indexed tree,) such that the index serves as indication of the element in the input array, and the actual element in the frequency table provides the amount of times that one number is seen (repeated) in the source collection. Then the new array is traversed again in another linear cost operation (this time dependent on the largest value contained in the input array,) and proceeds to update the frequencies of each element by adding its current frequency to the frequency of the element that came before it (even though the assymptotic cost is linear in nature, the computation involved starts at the second element, because the first element has no peer preceding it and thus has the "right" frequency assigned to it during construction of the frequency table.) Once this is done, the new array contains, at each index (recall the indices indicated the actual values contained in the input array) the amount of elements that are smaller than it (an implicit consequence is that even for indices referring to numbers not appearing in the input array, if some *smaller* number *did* appear in the array, they would have its frequency added to it.) This information already encodes the ordering of the element with the same index as the element with the same value as the index in the frequency table, and so the only thing left is to traverse the length of the original array, and index each of the elements of the frequency table with the yielded values of the input array, to get the position in which one must put the element indexed at the original input array into another output array (this output array is the second allocation that makes this algorithm non-in-place.)

Because I've yet to discuss the workings of the specific implementation that D. E. Knuth uses for the GraphBase sorting module, I've not yet decided on whether I should use the approach indicated in

[5], or otherwise follow through with the above approach. In terms of DS layout, it definitely seems like the former is better, but this is only a "gut feeling."

From rereading the initial documentation comments on the module, it seems the whole purpose of this sorting routine is not to perform a stable sort, but rather to purposefully shuffle elements that compare equal in random ways (i.e. to perform an intentional unstable sort by randomly laying out elements exposing partial equality,) while still ending the routine with an increasing sequence of elements layed out in the same linked list as the one exposed in [5].

For that, D. E. Knuth expects the users of the library functions to provide a structure that aligns with the requirements of the example structure `node` (that we already spoke of when commenting [5, Alg. 5.2.5R].) Because of `struct` layout constraints in C compilers (except when using `#pragma`s to change field packing behavior,) the fields used in the sorting routine, namely `key` and `link`, ought be the first two fields in the structures that the users provide to the library function. This limitation could be easily circumvented through codegen in Rust, and it would likely not affect compile-times. This is feasible because D. E. Knuth expects this sorting algorithm to be used selectively by certain algorithms that require adscribing order (or possible disorder if all `keys`' partial order compares equal) on some input data set, like the words contained in the `words.dat` file; Thus, the macro itself would be used on a case-by-case, (hopefully) fine-grained basis by library users. This may not be possible, based on the functions provided in the standard library for sorting elements in slices (`sort*()` and `sort_unstable*()`.) The variants of these functions that accept a closure expect to have an `Ordering` returned, which means that there must be at least a strict weak ordering relation between elements. If I try to manipulate the output based on a precomputed result of the `partial_cmp()` function, I would run into the issue of having to force a non-equal result between elements that, `key`-wise, would compare equal. Fast forward to the end of the sorting algorithm, and I would end up with a tangled mess of unordered `keys`, that have taken into account satellite data ordering as its primary resolution strategy, instead of `key` ordering.

To this extent, this single routine may be replaced with a regular, contiguous heap-memory allocated container like Rust's `Vec`, and instead of performing radix sort, performing a regular stable sort with control over resolution of the comparison function through a closure. This should allow the closure within the stable sort to perform the same random shuffling of values that evaluate to partial equality, and for any other value, resolve to the built-in total ordering of integral values.

The first part of the routine performing the initialization step **R1** in [5, Alg. 5.2.5R], may be potentially unsafe if the next returned random value covers the full unsigned integer range, namely 2^{31}, because 23 right-shifting operations are not enough to explicitly cover the range 0-255, which is the only valid range for the array that is being indexed with the returned (and post-bit shifted value of the linear

congruential random number engine.) The range covered is $[0, 256]$, but it ougth be $[0, 256)$ (note the segment is open at the end) to index the array without an out-of-bounds miss.

The last two passes of the algorithm are based on the MSD-pass idea proposed at the end of the [5, Sec. 5.2.5]. They follow the same principle as the one used for LSD passes. All passes also base their behavior off of the assumption that the keys will only ever hold some number with 6 digits tops, as the range of values for p is $[0, 6]$ (where p here has the same meaning as the one given by D. E. Knuth.)

This module will likely also be completely rewritten, as the only thing that it attempts to do efficently is to perform stable sorting with the effects of unstable sorting, making sure such effects are as non-deterministic in nature as possible. To that extent, it uses radix sort with the same linked list-like, and queue-like behavior as proposed in [5], but on the first two iterations of p considers randomly shuffled keys, which are then reordered into the desired layout. The goal is to compute the partial order of nodes in terms of their `key` fields to produce an increasing sequence in a contiguous container, while randomly shuffling values that compare equal. To allow further flexibility to library users, the module should be refactored into using a trait-based implementation with the same codegen idea as proposed with the `util` unions (commented at the end of Section 1.1.2) in the graph primitives. This should allow deriving the `PartialOrd` trait such that the satellite data remaining on the graph type determines the final ordering of the elements.

This should about do it with the `gb_sort.w` module. Before moving on to the generative routines, I believe it best to revisit the `gb_rand.w` module, as I didn't have the bibliographic references the author used to implement the linear congruential engine when I commented on the codebase.

1.1.5 Random number generation revisitted (`gb_flip.w`)

On the discussion about the potential unsafety of the sorting routines when calling the random number generator routine `gb_next_rand()`, the `gb_flip.w` module does mention that the range of returned numbers is bound to that of *signed*, and not *unsigned* integral values; Contrary to what I said, which included the range 2^{31}, the real range ends at $2^{31} - 1$, which does mean the eight most significant bits extracted on the first two runs of the partioning scheme are just fine, as the maximum value they map to is `0xFF`, which when shifted right by 23 bits, should yield a number in the expected range $[0, 256)$.

[5, Sec. 3.6] refers to older but analogous content to the one included in S. Skiena's catalogue under [2, Sec. 16.7]. The presented results are subpar compared with those presented by the more modern linear congruential engine formulas and heuristics discussed by the latter. Chief among the deficiencies of D. E. Knuth's methods is the period of the engine; 2^{55} on runs that don't hit the limitations commented on both the module file and Section 1.1.1 concerning the birthday spacings test. In contrast, S. Skiena

presents multiple better solutions known today, like the *Mersenne Twister* engine with a far larger period ($2^{19937} - 1$) and alternative implementations depending on the machine's word length.

The repeated cycling of the random number generation core routine (`gb_flip_cycle()`) is not backed by theoretical results, and [1, Sec. 3.6] makes note of this even when accounting for its apparent lack of defects. This likely implies that, in practice and across the decades (between the first publication of the chapter on random number generation in [1] back in the 1960s and the third edition published in the late 1990s,) D. E. Knuth likely experimented with the methods of the individual that came up with the idea, but that provided no formal proof of its correctness.

The random number generator module is getting completely replaced with another trait-based interface requiring of whichever methods one may need to compute random numbers in any general application under consideration within GraphBase. For testing purposes during development, the `rand` crate will be used instead. Once I get everything working, I may consider revisitting the algorithm in the original GraphBase and providing it as a `ClassicBackend` for the random number generation API.

1.1.6 Sorting routines revisitted (`gb_sort.w`)

Future efforts should focus on better understanding the sorting module, as the method explained in [5, Sec. 5.2.5] is clear, but the feasibility of implementing it in conjunction with a random number generator is not something I completely understand just yet. The purpose of this second exploration of the module should be evaluating whether it is feasible to provide the same functionality with a different implementation I understand better.

Upon further inspection of the documentation, it seems the module is not as limited as I initially believed it to be. It turns out the array `gb_sorted[]` exposed as part of the public interface is actually made up of linked lists, and not of the final records themselves as they are presented in [5, Alg. 5.2.5R, Alg. 5.2.5H]. Indeed, in the example code snippet given at the start of the module, the initial traversal over elements of the array is followed by a nested traversal through the nodes of each linked list, such that for each of the 127 linked DSs, a single pointer is stored to the starting node, that itself will hold a pointer to the next node in the corresponding list.

This would imply that if `gb_sorted[]` is to reflect the sorted order of all of the records (to use the terminology in [5, Sec. 5.2.5],) such that the linked list indexed at position 0 holds the first n smallest elements of the original linked list, then the initial partitioning is meant for both random shuffling and to separate all such elements in 256 linked lists, each an element of `alt_sorted[]`.

Another conclusion I've reached is that random shuffling is performed on a best-effort basis, as it's not enforced throughout sorting, but rather during the "initialization" (partitioning) steps. D. E. Knuth then puts his hopes into the initial sequence being sufficently shuffled for the next 4 passes of radix

sort to do their job just fine with respect to the `keys`, such that the satellite data of `keys` that compare equal is left in the same position as given post-random shuffling thanks to the stability of radix sort.

Two paragraphs above, the use of a general bound n for the number of elements in one of the linked lists in the sorted array of linked lists was not defined. To determine the index of the specific linked list in which a `key` or `key` range lies in, one may consider the following bounds on the value of the `key`(s) in relation with the valid range of indices into the array.

$$\begin{aligned}&\text{For index } 0 \leq j < 256, \text{linked list } \texttt{gb_sorted}[j] \\ &\text{contains } j \cdot 2^{24} \leq \texttt{key} < (j+1) \cdot 2^{24}.\end{aligned} \tag{1.4}$$

Both initial random shuffling and later sorting can be reimplemented in terms of a simpler scheme, that doesn't lose any of the theoretical performance of 6 passes of radix sort with radix 256. Whether Rust has a method equivalent to the shuffling algorithm in C++ is something else, but if the standard library doesn't provide one, we can always look at the implementation of libcxx or glibc for reference. In terms of sorting, much like radix sort, allocations beyond that of the input collection cannot be avoided because we need stable sorting to avoid reordering the elements that were randomly shuffled initially, so using the built-in `sort_unstable()` with ipnsort should do just fine. Alternatively, if the source of randomness can be determined to be the hardware generator at compile time, then a `cfg` flag for conditional compilation should allow using an unstable sort algorithm, because any possible reordering not based on the fields that define the partial ordering relation should resolve differently on every run, as the satellite data has been shuffled differently. The `std::random_device` already available in the C++ standard library could do this without having to use any form of conditional compilation, but it remains to be known whether the `rand` crate implements some such functionality.

1.2 On the book

The first chapter explains the use of the non-kernel modules, and contrary to what my initial beliefs, does not use `gb_basic.w` as the building blocks for the rest of the programs. If anything, starting from the `gb_words.w` module, all other generative routines are increasingly complex and often involve the use of (increasingly) more complex algorithms, like the *simplex* optimization algorithm for producing some of the graphs in `gb_basic.w` (which ironically is not as basic in nature as I thought it to be.)

A good starting point may be to review again the comments on the viability of some of these routines mentioned at the start of the chapter, as D. E. Knuth indicates that some of them are not a good fit for real use outside demonstration purposes.

[3, Ch. 2] follows with an explanation of the internal kernel files that I already went through, and so may prove to be more immediately useful to the existing notes taken on those modules than chapter 1 does now.

Upon rereading the material at the start of the chapter, it seems that the generator modules are all potentially capable of being used in non-trivial settings; It is the demonstration programs with names missing the `gb` prefix that should be considered as exploratory in nature and not as full showcases of the potential of the generative routines.

The `gb_save.w` module is an exception to the above statements on the complexity of the invovled theory, as the routines are intended for the purposes of saving in a standard format some graph produced by the generative routines. This module likely defines a more restrictive grammar than that of the `gb_io.w` module, as the latter only required of parsing in the first 4 lines and the last line of the input data set files, the rest was free-form and thus only resolved through lexer-like routines. Because the I/O interface is likely going to be the most generic API in the rewrite, maybe it proves to be a good starting point to implement one of the generative modules, and immediately proceed to the implementation of the output routines, as that should allow better understanding the parser logic and possibly seeing whether the final program should include the "old" logic for backwards-compatible purposes, but rely mostly on the use of modern serialization practices like those in the `serde` crate when attempting to distribute graphs between users of the library.

An alternative route would be to implement the kernel core routines, follow up with the `gb_basic.w` generative module, and then continue with the `gb_save.w` module. D. E. Knuth himself recommends persuing this module right after an initial read of the kernel routines.

[3, Sec. 2.1] does mention that the main purpose of the memory strategy behind using `Areas` is to have all allocations pertinent to some `Graph` (and in the off chance any other type) be centralized under a common umbrella that would allow the user of the routines to more easily manage the resources alloacted on the heap during creation and potential destruction of each of the elements associated with their implementations. This is pretty much RAII, but back when not even C++ implemented the paradigm in some its types. [3, Sec. 3.7] mentions that the demo program `queens.w` should serve as a good introduction to writing programs that use the GraphBase library, so that might also be something to look into.

[3, Ch. 4] alludes to the fact the source files contained at the end of the book are actually the same CWEB files that are included with the source files of the distribution, but additionally weaves into their DVI formats, so that should provide some insights into their workings (outside a termianl pager view,) considering the full GraphBase is documented.

The new roadmap looks as follows: (1) read through the kernel files's weaved result in [3, Sec. *Programs of the Stanford GraphBase*] and possibly take notes of missing material, (2) read through the same weaved contents but for the `gb_basic.w` module, and, (3) read through the `gb_save.w` module. The latter two are also going to require taking notes on them, but that is obvious because no notes have been taken just yet. Once all that is done, implementation details should start being discussed.

1.2.1 GB_FLIP

Contrary to my initial beliefs, the random number generator is conjectured to be capable of resolving to a period of potentially $2^{85} - 2^{30}$, except for one input seed value. Even though the initial implementation was determined to use the standard `rand` crate in the Rust ecosystem, this may prove useful while implementing the code in the `ClassicBackend` of the random number generator API.

A neat trick that I noted in Section 1.1.1, but that I didn't quite make a good point of, was that D. E. Knuth uses the bitwise `&` operation to compute the modulo between two numbers. This may prove useful at some point down the line, though maybe the built-in modulo operation is laready optimized to produce good, possibly bitwise operations in the underlying Rust runtime if it deems the situation fit for it.

Beyond this, my conclusions continue being the same as those in Section 1.1.1. The initial implementation is only goin to use the `rand` crate as that can likely also be configured with the right set of parameters to have the seed be deterministic, as D. E. Knuth himself recommends and very much expects to produce initially random but regardless reproducible results across runs with the same seed.

1.2.2 GB_GRAPH

Thinking again the approach taken with the `verbose` flag that is exposed to outside programs, I can say that this could greatly benefit from either not including, or using the `tracing` crate instead. This should allow providing some degree of verbosity to the user of the library as a (possibly) opt-in feature when debugging, or to plug their own verbose options by using some exposed interface that toggled verbosity in certain places.

This is mostly due to the fact kernel modules in the original GraphBase, as conceived by D. E. Knuth, were not expected to be used outside the purposes of GraphBase programs, and because C was severely constrained at the time when it came to logging capability interop with programs using some library. Either way, in Rust it would be more idiomatic to let the programs using the library crate have whatever shenanigans they got going with something like `clap` for argument parsing, and `log` or `color_eyre` for error handling to do the heavywork of presenting the contents to the user.

A crate-wide macro to toggle logging functionality would be a good addition, if not a necessary one when testing programs outside the library. The initial implementation, though, is likely not to require this feature so **I'm leaving it in the backburner**.

Contrary to my initial comments on the offsetting of values for error codes expanding to numbers 10 and 11, these are not meant to have some number (offset) added to them, but rather to indicate that something else, covered by `io_errors`, has happened. Thus if upon inspecting `panic_code`, the user finds any one of these, they should be lead to believe that further information is embedded within any one of the static globals concerning I/O-bound errors.

From looking again into the definition of `Arc`, maybe it's a good idea to get out of the standard fields the field indicating the length of the arc, as that could be included through the complementary information that the codegen alternative to using `union`s would provide. This should only be decided, though, once the fields most often used by the generative routines are evaluated, and thus once I can say for sure that this is a good idea (otherwise I doubt D. E. Knuth didn't notice that the length of each arc in an embedded graph couldn't have been added to the utility fields instead, considering these also cover, under the declaration of the `util` union type, an integer of the same width as that used for the `len` field of `Arc`.)

The same as above also applies to the field containing each of the fields in the `Vertex` type. This may require further work in its implementation, as the hashing scheme to allow for $O(1)$ lookup into the vertices of a given graph `g` relies on always having available the vertex "name" associated with each instance of a `Vertex`. An alternative would be to have the `Hash` trait implementation be derived only on the types that specified such an option through the proc-macro that would get exposed in the public interface.

This, though, also relies on outer attribute-like macros being resolved post-tokenization and *before* parsing and expansion of the `derive`-like macros; Otherwise, the whole thing falls apart, as `derive` macros are not meant to be inert. [8] speaks of attribute-like macros expanding in the order in which the lexer detects the attributes and passes them off the the tokenization routines, but it doesn't specify the order in which some macro gets evaluated if a prior macro generated another, pre-expansion-wise nonexisting, macro at the end of its own invocation. It's quite likely the generated tokens will run as well, based on the fact the reference refers to all "lower" items as being fed the entirety of the token tree in the span of the scope-level item, which implies that a top-level macro must include as part of its generated output the prior macros for them to expand. This sounds like it is equivalent to having the macro generate a new macro, and not just regenerate an existing macro. Whenever the function tagged with the corresponding `proc_macro*` attribute returns or panics, whatever codegen

is inserted into the "source file" (already loaded into memory by the compiler,) is the same regardless of whether it is an existing macro or a new macro. But testing is pending.

The `Hash` trait cannot be derived, because each `Vertex` is likely to hold information beyond that of its string identifier, and the default implementation of the `derive` macro on `Hash` is implemented in terms of calling `hash()` on each of the fields of the derived type; But GraphBase only requires hashing in terms of the string field, and not in terms of a pointer, capacity and length for the adjacency lists of each of the vertices. A possible solution would be to implement another `derive`-like proc-macro that would base its implementation off of the presence of a type implementing `AsRef<&str>` or `Deref<&str>`, such that it could be applied internally to any new graph types resulting from the rest of the codegen proc-macros.

Reading about the memory allocation strategy in the weaved document seems clearer now. The comments on it were not in the wrong at all, and it's quite possibly going to get removed from the Rust implementation. An alternative would be for some future backend (definetely not in the first implementation) to use `MaybeUnint` to replicate the behavior achieved through memory `Area`s in the original GraphBase. An addition to this would be to mediate between the library user and the requests to the global allocator by building up some `Layout` from the `std::alloc` module to have the amount of memory "customized." Though, then again, this is only a future plan, as the initial implementation is only going to use the built-in OBRM that Rust already implements.

On the previous comments about the size of the graph primitive types, my comments also apply to the `Graph` type. Looking at it again, it seems more and more like the only field that is truly representative of anything in a graph is the DS that is used to represent the vertices and the arcs. The strictly mathematical definition is $G = (V, E)$ after all, and all other fields should be codegenerated with the apropriate methods/associated functions if need be. The only field that may be worth keeping around would be the UID of the graph; As per the original GraphBase, it is not unique in nature, but this could very well change in the Rust rewrite if the generative routines prove to fit well with a unique ID instead of a function-call-based ID. This change would also discard the routines for generating both regular IDs and compound IDs.

This could lead to an implementation that takes the codegen idea in two ways. On the one hand, we would consider a very primitive data type for each of the core graph types, and a proc-macro would extend with knowledge at compile time of the desired extension type some new implementations on top of that primitive. On the other hand, the graph primitives exposed to the user would also consider another proc-macro that would, on the user's own type, implement the field they want, and expose the entire API of the graph primitive, itself derived from one of the possible codegen paths exposed through the prior, internal proc-macro as additional options of the public interface proc-macro.

The graph building routine `gb_new_graph()` mentions that the space reserved is of `n + extra_n`, instead of the parameterized $n = |V|, G = (V, E)$. On the comments in Section 1.1.2, I mentioned that space was both allocated and assigned to the `Area` of the `Graph`, but that is not the case. Memory is assigned for n + `extra_n` vertices, but only the first n vertices are initialized with the null string. The othe `extra_n` vertices are only part of the same memory `Area` pointed to by the `first` field of the graph's `data` field, but are not explicitly initialized to the expected state of unused vertices. **In Rust, this would translate to keeping a vector and reserving for the specified capacity, without resizing the actual size of the collection on vertex creation.**

All of the routines for allocating arcs and having them either assigned to a directed or undirected graph are getting removed in the rewrite. They should work without special treatment if we use the built-in capabilities of the `Vec` collection in Rust. The `gb_virgin_arc()` is one of the routines that is especially not required, considering there is no need to keep track of the data allocated on the heap manually; RAII will do it for us.

The scheme that D. E. Knuth follows to have the arcs in a directed graph be contiguous in the heap-allocated array containing the data for both arcs and strings in a given graph cannot be easily replicated in Rust, so it's going to have to get replaced with some other mechanism. A possible alternative would be to have the arcs be shared by some global resource handle that had them grouped between vertices of the same graph, such that the arcs held in the adjaceny matrix were only pointers (references) to the edges owned by the overarching resource handle. Maybe the resource handle could be made into being part of the `Graph` type, such that the "regular" layout of an adjacency list is upkept on a `Vertex`-per-`Vertex` basis, but the contents of each of those vertices underlying linked lists (possibly implemented as either one of a contiguous collection or a double-ended queue,) would be references to the resource handle. Even though we speak of a resource handle because the memory would be owned by whatever container would have the memory allocated for arcs, it's quite possibly going to be abstracted in more graph-logic-consistent manner.

The allocations for strings can safely rely on raii so they can be stored within the same `Vertex` record, and the `Drop` implementation once they go out of scope should trigger memory freeing.

The need for D. E. Knuth to implement the graph switching routine only arised as a limitation of the time when he implemented GraphBase, so it's getting replaced with associated functions in the overarching types making up the set of graph primitives in the rewrite. This should also allow replacing the `gb_new_arc()` and `gb_new_edge()` with the same set of arc/edge addition routines, instead of reimplementing them as methods *and* free functions.

1.2.3 GB_IO

The only thing that was not noted in the previous comments on this set of routines is the use of the `fill_buf()` routine, which attempts to bridge the gap with systems that add whitespace padding to be conformant with their filesystem requirements. This could still be a limitation that needs to be addressed, even with today's devices. Still, this particular feature is going to require more research into which modern-day FSs use byte padding in user files, and it's not getting into the initial release.

Bibliography

[1] D. E. Knuth, *Seminumerical Algorithms*, 3rd ed., vol. 2. in The Art of Computer Programming, vol. 2.

[2] S. Skiena, *The Algorithm Design Manual*, 3rd ed.

[3] D. E. Knuth, *The Stanford GraphBase*.

[4] D. E. Knuth, *Literate Programming*.

[5] D. E. Knuth, *Sorting and Searching*, 2nd ed., vol. 3. in The Art of Computer Programming, vol. 3.

[6] N. Deo, in *Handbook of Data Structures and Applications*.

[7] T. H. Cormen, C. E. Leiserson, R. R. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed.

[8] *The Rust Reference - 1.92.0 - Stable*.